Linear Mixed Models (LMMs) for Labelled Proteomics

Christophe Vanderaa Lieven Clement

This chapter builds upon the introduction to linear mixed models. We will here cover more advanced concepts using `msqrob2`. To illustrate these advanced concepts, we will use the spike-in study published by Huang et al. (2020). We chose this data set because:

1. Spike-in data contain ground truth information about which proteins are differentially abundant, enabling us to show the impact of different analysis strategies.
2. It has been acquired with a TMT-labelling strategy that require a complex experimental design. This provides an excellent example to explain the different sources of variability in an MS experiment and demonstrate the flexibility of `msqrob2` to model this variability.

Background

Labelling strategies in mass spectrometry (MS)-based proteomics enhance sample throughput by enabling the acquisition of multiple samples within a single run. The labelling strategy that allows the highest multiplexing is the tandem mass tag (TMT) labelling and will be the focus of the current tutorial.

TMT workflow

TMT-based workflow highly overlap with label-free workflows. However, TMT-based workflows have an additional sample preparation step, where the digested peptides from each sample are labelled with a TMT reagent and samples with different TMT reagents are pooled in a single TMT mixture. The signal processing is also slightly affected since the quantification no longer occurs in the MS1 space but at the MS2 level. It is important to understand that TMT reagent are isobaric, meaning that the same peptide with different TMT labels will have the same mass for the intact ion, as recorded during MS1. However, the TMT fragments that are released upon fragmentation during MS2, also called TMT reporter ions, have label-specific masses. The TMT fragments have an expected mass and are distributed in a low-mass range

of the MS2 space. The intensity of each TMT fragment is directly proportional to the peptide quantity in the original sample before pooling. The TMT fragment intensities measured during MS2 are used as quantitative data. The higher mass range contains the peptide fragments that compose the peptide fingerprint, similarly to LFQ. This data range is therefore used for peptide identification. Interestingly, the peptide fingerprint originates from the same peptide across multiple samples. This leads to a signal boost for low abundant peptides and hence should improve data sensitivity and consistency.

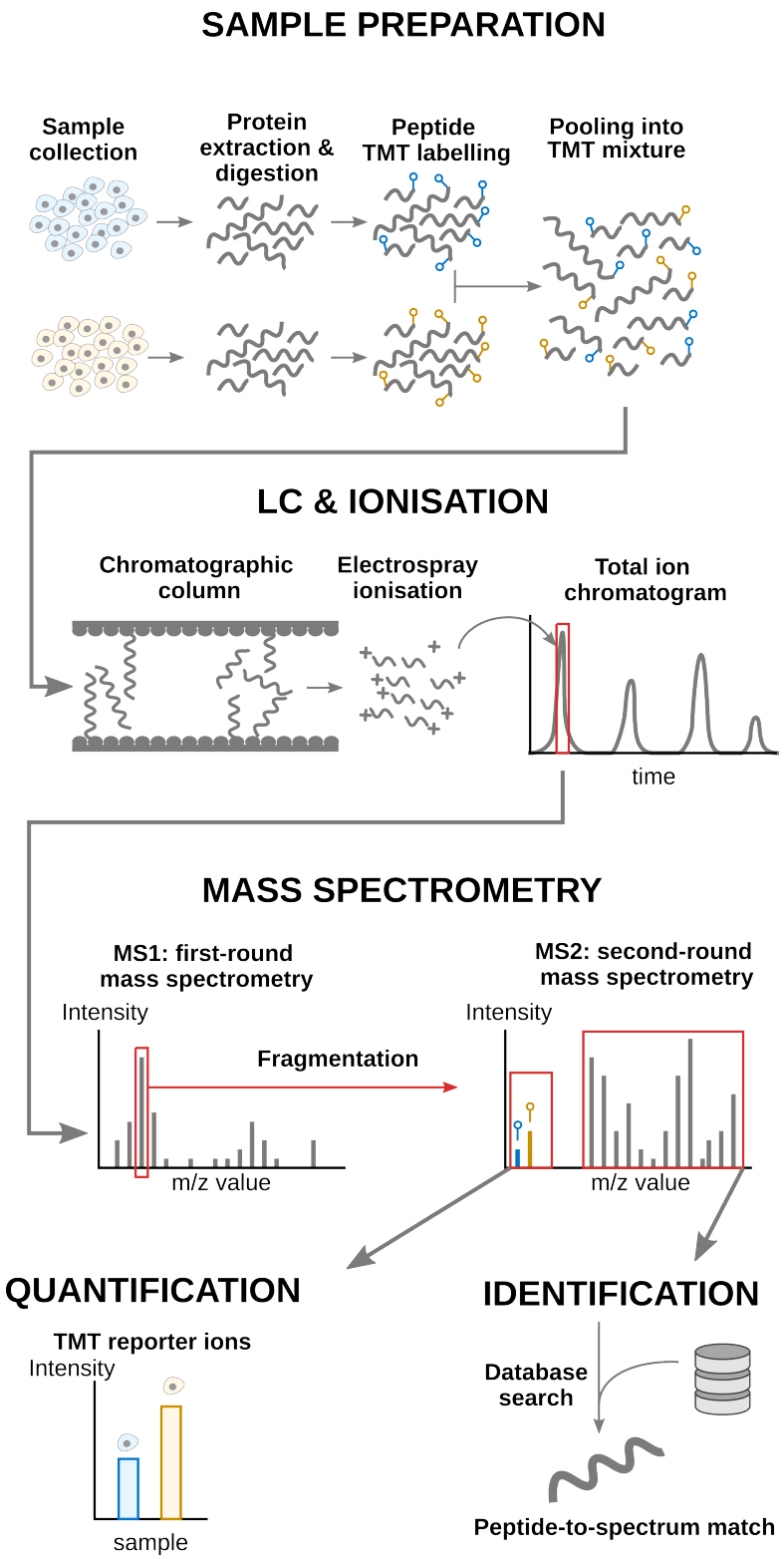


Figure 1: Overview of an TMT-based proteomics workflow.

Challenges

The analysis of TMT-based proteomics data shares the same challenges as the data analysis challenges for LFQ:

- MS-based proteomics doesn't measure proteins directly, but their constituting **peptide ions**. The protein-level information needs to be reconstructed from the ion data. In this tutorial, we will start from the peptide data, which has been constructed from the ion data by MaxQuant.
- All peptides cannot be ionised with the same efficiency. Poor ionisation will lead to reduced signal as less ions will hit the detector, hence leading to a huge variability in intensity among different peptide species, even when they originate from the same protein.
- The identification step is not trivial and prone to errors. PSM misidentification leads to the assignment of a quantitative values from another peptide with likely another ionisation efficiency and relative abundance. Hence this misassigned value will become an outlier.
- Moreover, the ion selection for MS2 depends on its intensity (recall that only the top most intense ion peaks are send for MS2). Therefore, the chance to measure and, subsequently, identify a peptide will depend on its abundance. Non identified peptides will lead to data missingness, which is related to the underlying quantification value. This phenomenon is known as missingness not at random. Next to that, many reasons can lead to ions not being selected or identified irrespective of their quantification value leading to missingness that is not related to its quantitative value. This is referred to as missingness completely at random. The missingness issue is not negligible: only 41% of all proteins are quantified across all samples, and the number drops to 6.6% when considering peptides.
- The identification issues lead to unbalanced peptide missingness across samples, and the patterns of missing values are potentially different for every peptide, highlighting the need for an automatised solution that is robust against missing values.
- Technical variations during the experiment can lead to systematic fluctuations across samples. The most obvious reason is when different sample amounts are injected into the instruments, due to small pipetting inconsistencies for instance. However, these differences lead to unwanted variation that should be discarded when answering biological questions.
- TMT workflows impose an additional challenge. Contemporary experiments often involve increasingly complex designs, where the number of samples exceeds the capacity of a single TMT mixture, resulting in a complex correlation structure that must be addressed for accurate statistical inference. We will describe in the modelling section the different sources of variation.

Experimental context

The data set used in this chapter is a spike-in experiment (PXD0015258) published by Huang et al. (2020). It consists of controlled mixtures with known ground truth. UPS1 peptides at concentrations of 500, 333, 250, and 62.5 fmol were spiked into 50 g of SILAC HeLa peptides, each in duplicate. These concentrations form a dilution series of 1, 0.667, 0.5, and 0.125 relative to the highest UPS1 peptide amount (500 fmol). A reference sample was created by combining the diluted UPS1 peptide samples with 50g of SILAC HeLa peptides. All dilutions and the reference sample were prepared in duplicate, resulting in a total of ten samples. These samples were then treated with TMT10-plex reagents and combined before LC-MS/MS analysis. This protocol was repeated five times, each with three technical replicates, totaling 15 MS runs.

We will start from the PSM data generated by Skyline and infer protein-level differences between samples. To achieve this goal, we will apply an msqrob2TMT workflow, a data processing and modelling workflow dedicated to the analysis of TMT-based proteomics datasets (Vandenbulcke et al. 2025). We adapted the data preprocessing to the default workflow of this course, i.e. median of ratios normalisation and maxLFQ summarisation, both performed run by run. We will demonstrate how the workflow can highlight the spiked-in proteins. Before delving into the analysis, let us prepare our computational environment.

Load packages

We load the `msqrob2` package, along with additional packages for data manipulation and visualisation.

```
library("msqrob2")
library("dplyr")
library("ggplot2")
library("patchwork")
```

We also configure the parallelisation framework.

```
library("BiocParallel")
register(SerialParam())
```

Data

The data have been deposited by the authors in the `MSV000084264` MASSiVE repository, but we will retrieve the time stamped data from our Zenodo repository. We need 2 files: the Skyline

identification and quantification table generated by the authors and the sample annotation files.

To facilitate management of the files, we download the required files using the `BiocFileCache` package. The chunk below will take some time to complete the first time you run it as it needs to download the (large) file locally, but will fetch the local copy the following times.

```
library("BiocFileCache")
bfc <- BiocFileCache()
psmFile <- bfcrpath(bfc, "https://zenodo.org/records/14767905/files/spikein1_psms.txt?downloa
annotFile <- bfcrpath(bfc, "https://zenodo.org/records/14767905/files/spikein1_annotations.cs
```

Now the files are downloaded, we can load the two tables.

PSM table

An MS experiment generates spectra. Each MS2 spectra are used to infer the peptide identity thanks to a search engine. When an observed spectrum is matched to a theoretical peptide spectrum, we have a peptide-to-spectrum match (PSM). The identification software compiles all the PSMs inside a table. Hence, the PSM data is the lowest possible level to perform data modelling.

Each row in the PSM data table contains information for one PSM (the table below shows the first 6 rows). The columns contains various information about the PSM, such as the peptide sequence and charge, the quantified value, the inferred protein group, the measured and predicted retention time and precursor mass, the score of the match, … In the case of TMT data, the quantification values are provides in multiple columns (start with `"Abundance."`), one for each TMT label. Regardless of TMT or LFQ experiments, the PSM table stacks the quantitative values from samples in different runs below each other.

```
psms <- read.delim(psmFile)
qcols <- grep("Abundance", colnames(psms), value = TRUE)
```

Identifier(s)	PSM	Modified AMS	Mass tertiary	Mass PRM	Protein Accession	Description	Mass Error [ppm]	Search Engine	Activation	Isolation Interference [%]	Spectrum Abundance	Spectrum File
Cotrifyguated (Oligous)	Heglasnlukpsvlqqlpqtlqqlqtlqqlqtlqql

Identifier	Modification	Mass	Mastertemplate	Protein	Accession	Description	Mixture	Isolation	Active	Interference	Spectra
Certify Checking	Modification Notes	Mass Measurement	Mastertemplate Instructions	Protein Instructions	Accession Changes	Description Range	Mixture Ratio	Isolation Time	Active Duration	Interference Pattern	Spectra Files
Failed (F2)	High-collected (F2)	[R]-EHQVMQ[Oxidation]PQ[Deamidation]PQ	Term(TMT6plex); M8(Oxidation); R9(Label:13C(6)15N(4))	POU domain-containing octamer-binding protein OS=Homo sapiens GN=PO5F1 PE=1 SV=4	POU domain-containing octamer-binding protein OS=Homo sapiens GN=PO5F1 PE=1 SV=4	0.52	0.04	1	1	359.1	0.00002	Ab-initio	Ab-initio	Ab-initio

| Identifier | PSM | Modified sequence | Mass error [ppm] | Missed cleavages | Proteins | Leading razor protein | Protein accessions | Decoy | Contaminant | Score | Delta score | Combinatorics | Intensity | Reverse | Potential contaminant | id | Evidence IDs | Best localization probability | Best localization score | Best localization delta score | Localization probability | Localization score | Localization delta score | Localization probabilities | Localization scores | Localization delta scores | Acetyl (Protein N-term) | Oxidation (M) | Acetyl (Protein N-term) site IDs | Oxidation (M) site IDs | Isolation interference [%] | Acetyl (Protein N-term) probabilities | Oxidation (M) probabilities | Acetyl (Protein N-term) score diffs | Oxidation (M) score diffs | Fragmentation | Mass analyzer | Type | Scan event number | Isotope index | Simple mass error [ppm] | Simple mass error [Da] | Retention time | Retention length | Retention length (FWHM) | Calibrated retention time | Calibrated retention time start | Calibrated retention time finish | Retention time calibration | Match time difference | Match q-value | Match score | Number of data points | Number of scans | Number of isotopic peaks | PIF | Fraction of total spectrum | Base peak fraction | PEP | MS/MS count | MS/MS scan number | Score for MS/MS | Localization of MS/MS | Raw file | Experiment | Fraction | Multiplicity | Charge | m/z | Mass | Resolution | Uncalibrated - Calibrated m/z [ppm] | Uncalibrated - Calibrated m/z [Da] | MS/MS m/z | Uncalibrated mass error [ppm] | Uncalibrated mass error [Da] | Max intensity m/z 0 | Mass precision [ppm] | Mass precision [Da] | Base peak intensity | Total ion current | Collision energy | Ion injection time | Total ion current (TIC) | Base peak m/z | Base peak mass | Ion mobility index | Ion mobility index length | Ion mobility index length (FWHM) | Ion mobility value | Ion mobility value start | Ion mobility value finish | Ion mobility data type | Ion mobility type | Ion mobility unit | Ion mobility precursor m/z | Ion mobility precursor mass | Ion mobility precursor charge | Ion mobility precursor intensity | Ion mobility precursor intensity fraction | Ion mobility precursor intensity coverage | Ion mobility precursor intensity apex fraction | Ion mobility precursor intensity apex position | Ion mobility precursor intensity apex position fraction | Ion mobility precursor intensity apex position start | Ion mobility precursor intensity apex position finish | Ion mobility precursor intensity apex position start fraction | Ion mobility precursor intensity apex position finish fraction | Ion mobility precursor intensity skewness | Ion mobility precursor intensity kurtosis | Ion mobility precursor intensity symmetry | Ion mobility precursor intensity symmetry left | Ion mobility precursor intensity symmetry right | Ion mobility precursor intensity symmetry left fraction | Ion mobility precursor intensity symmetry right fraction | Ion mobility precursor intensity symmetry left start | Ion mobility precursor intensity symmetry left end | Ion mobility precursor intensity symmetry right start | Ion mobility precursor intensity symmetry right end | Ion mobility precursor intensity symmetry left start fraction | Ion mobility precursor intensity symmetry left end fraction | Ion mobility precursor intensity symmetry right start fraction | Ion mobility precursor intensity symmetry right end fraction | Ion mobility precursor intensity symmetry left slope | Ion mobility precursor intensity symmetry right slope | Ion mobility precursor intensity symmetry left slope fraction | Ion mobility precursor intensity symmetry right slope fraction | Ion mobility precursor intensity symmetry left slope start | Ion mobility precursor intensity symmetry left slope end | Ion mobility precursor intensity symmetry right slope start | Ion mobility precursor intensity symmetry right slope end | Ion mobility precursor intensity symmetry left slope start fraction | Ion mobility precursor intensity symmetry left slope end fraction | Ion mobility precursor intensity symmetry right slope start fraction | Ion mobility precursor intensity symmetry right slope end fraction | Ion mobility precursor intensity symmetry left slope fraction | Ion mobility precursor intensity symmetry right slope fraction

Sample annotation table

Each row in the annotation table contains information about one sample. The columns contain various descriptors about the sample, such as the name of the sample or the MS run, the treatment (here the spike-in condition), the lab that acquired the sample or any other biological or technical information that may impact the data quality or the quantification. Without an annotation table, no analysis can be performed. The annotation table used in this tutorial has been generated by the authors.

```
coldata <- read.csv(annotFile)
```

We perform a little cleanup for concise output, and rename the `Channel` column to `Label` because it contains information about the TMT label used.

```
coldata <- coldata[, c("Run", "Channel", "Condition", "Mixture", "TechRepMixture")]
coldata$File.Name <- coldata$Run
coldata$Run <- sub("^.*(Mix.*).raw", "\\1", coldata$Run)
colnames(coldata)[2] <- "Label"
```

Run	Label	Condition	Mixture	TechRepMixture	File.Name
Mixture1_01	126	Norm	Mixture1	1	161117_SILAC_HeLa_UPS1_TMT10_Mixture1_01.raw
Mixture1_01	127N	0.667	Mixture1	1	161117_SILAC_HeLa_UPS1_TMT10_Mixture1_01.raw
Mixture1_01	127C	0.125	Mixture1	1	161117_SILAC_HeLa_UPS1_TMT10_Mixture1_01.raw
Mixture1_01	128N	0.5	Mixture1	1	161117_SILAC_HeLa_UPS1_TMT10_Mixture1_01.raw
Mixture1_01	128C	1	Mixture1	1	161117_SILAC_HeLa_UPS1_TMT10_Mixture1_01.raw
Mixture1_01	129N	0.125	Mixture1	1	161117_SILAC_HeLa_UPS1_TMT10_Mixture1_01.raw

Create the QFeatures object

We use `readQFeatures()` to create a `QFeatures` object. Since we start from the PSM-level data, the approach is somewhat more elaborate. You can find an illustrated step-by-step guide in the QFeatures vignette. First, recall that every quantitative column in the PSM table contains information for multiple runs. Therefore, the function split the table based on

the run identifier, given by the `runCol` argument (for Skyline, that identifier is contained in `Spectrum.File`). So, the `QFeatures` object after import will contain as many sets as there are runs. Next, the function links the annotation table with the PSM data. To achieve this, the annotation table must contain a `runCol` column that provides the run identifier in which each sample has been acquired, and this information will be used to match the identifiers in the `Spectrum.File` column of the PSM table. The annotation table must also contain a `quantCols` column that tells the function which column in the PSM table contains the quantitative information for a given sample. In this case, the `quantCols` depend on

```
coldata$runCol <- coldata$File.Name
coldata$quantCols <- paste0("Abundance..", coldata$Label)
(spikein <- readQFeatures(
  psms, colData = coldata,
  runCol = "Spectrum.File",
  quantCols = qcols
))
```

```
  |
  |                                                                      |   0%
  |
  |=====                                                                 |   7%
  |
  |=========                                                             |  13%
  |
  |==============                                                        |  20%
  |
  |===================                                                   |  27%
  |
  |=======================                                               |  33%
  |
  |============================                                          |  40%
  |
  |================================                                      |  47%
  |
  |====================================                                  |  53%
  |
  |========================================                              |  60%
  |
  |============================================                          |  67%
  |
  |================================================                      |  73%
```

```
  |
  |=======================================================               |  80%
  |
  |=========================================================             |  87%
  |
  |===============================================================       |  93%
  |
  |======================================================================| 100%
```

```
An instance of class QFeatures (type: bulk) with 15 sets:

 [1] 161117_SILAC_HeLa_UPS1_TMT10_Mixture1_01.raw: SummarizedExperiment with 1905 rows and 10
 [2] 161117_SILAC_HeLa_UPS1_TMT10_Mixture1_02.raw: SummarizedExperiment with 1902 rows and 10
 [3] 161117_SILAC_HeLa_UPS1_TMT10_Mixture1_03.raw: SummarizedExperiment with 1952 rows and 10
 ...
 [13] 161117_SILAC_HeLa_UPS1_TMT10_Mixture5_01.raw: SummarizedExperiment with 1919 rows and 1
 [14] 161117_SILAC_HeLa_UPS1_TMT10_Mixture5_02.raw: SummarizedExperiment with 1909 rows and 1
 [15] 161117_SILAC_HeLa_UPS1_TMT10_Mixture5_03.raw: SummarizedExperiment with 1844 rows and 1
```

We now have a `QFeatures` object with 15 sets, each containing data associated with an MS run. The name of each set is defined by the name of the corresponding file name of the run, which is unnecessarily long. We simplify the set names, although this step is optional and only meant to improve the clarity of the output.

```
## This is optional
names(spikein) <- sub("^.*(Mix.*).raw", "\\1", names(spikein))
(inputNames <- names(spikein))
```

```
 [1] "Mixture1_01" "Mixture1_02" "Mixture1_03" "Mixture2_01" "Mixture2_02"
 [6] "Mixture2_03" "Mixture3_01" "Mixture3_02" "Mixture3_03" "Mixture4_01"
[11] "Mixture4_02" "Mixture4_03" "Mixture5_01" "Mixture5_02" "Mixture5_03"
```

Data preprocessing

We will follow a similar data processing workflow as before.

Encoding missing values

Any zero value needs to be encoded by a missing value.

```
spikein <- zeroIsNA(spikein, inputNames)
```

Log2 transformation

Log2-transformation solves the heteroskedasticity issue, but also provides a scale that directly relates to biological interpretation.

```
logNames  <- paste0(inputNames, "_log")
spikein <- logTransform(
    spikein, inputNames, name = logNames, base = 2
)
```

Sample filtering

We remove the reference samples since `msqrob2` workflows do not use reference normalisation.

```
spikein <- subsetByColData(spikein, spikein$Condition != "Norm")
```

PSM filtering

Filtering removes low-quality and unreliable PSMs that would otherwise introduce noise and artefacts in the data. Conceptually, PSM filtering is identical to peptide filtering, but we will here apply filtering criteria for which some are not readily available in the data. Therefore, we will add custom filtering variable to the `rowData` that will then be used with `filterFeatures()`. This provides an ideal use case to demonstrate the customisation of a filtering workflow.

Remove ambiguous identifications

The background proteins originate from HeLa cells, which also contain UPS proteins. The background UPS proteins and the spiked-in UPS proteins differ in metabolic labelling, so we should be able to distinguish them. We used the PSM-level data searched with mascot, as provided by the MSstatsTMT authors who used two mascot identification nodes. In one node they searched the SwissProt database for proteins with static modifications related to the metabolic labelling, in the other node they searched the Sigma_UPS protein database without these static modifications. Ideally, this should separate the spiked-in UPS proteins and the

UPS proteins from the HeLa cells, however, this is not always the case. The SwissProt search is expected to return peptide-spectrum matches (PSMs) for all proteins, including non-UPS HeLa, UPS HeLa, and spike-in UPS proteins. Conversely, the Sigma_UPS search is expected to return PSMs exclusively for spike-in UPS proteins. However, a PSM that matches a UPS protein in the SwissProt search but is not identified as such in the Sigma_UPS search could either correctly originate from a HeLa protein or represent a spiked-in UPS protein that was not recognised as such in the Sigma_UPS search. Additionally, there are ambiguous PSMs that are not matched to a UPS protein in the HeLa search but are matched to a UPS protein in the SwissProt search. To address this, we exclude these ambiguous proteins from the analysis.

To define the amibiguous PSMs, we retrieve the PSM annotations from the `rowData` and create a new colum indicating whether a PSM belongs to a UPS protein or not, based on the protein SwissProt identifiers. For this, we apply a custom filtering worklow:

1. **Collect data**: combine all the `rowData` information in a single table. We will apply the filter on the

```
rowdata <- rbindRowData(spikein, logNames)
```

2. **Compute new variable**: (2a) define whether the PSM's protein group is a UPS protein and then (2b) define an ambiguous PSM as a PSM that is marked as UPS by the SwissProt identifier but not by the Sigma_UPS node (`Marked.as` column), and inversely.

```
## 2a.
rowdata$isUps <- "no"
isUpsProtein <- grepl("ups", rowdata$Protein.Accessions)
rowdata$isUps[isUpsProtein] <- "yes"
## 2b.
rowdata$isUps[!isUpsProtein & grepl("UPS", rowdata$Marked.as)] <- "amb"
rowdata$isUps[isUpsProtein & !grepl("UPS", rowdata$Marked.as)] <- "amb"
```

3. **Reinsert in the rowData**: insert the modified table with new information back in the `rowData` of the different sets. This means that the single table with `rowData` information needs to be split by each set. `split()` will produce a named list of tables and each table will be iteratively inserted as `rowData` of the set.

```
rowData(spikein) <- split(rowdata, rowdata$assay)
```

4. **Apply the filter**: the filtering is performed by `filterFeatures()` using the new information from the `rowData`. We specify `keep = TRUE` because the input sets (before log-transformation) do not contain the filtering variable, so we tell the function to keep all PSMs for the sets that don't have the variable `isUps`.

```
spikein <- filterFeatures(spikein, ~ isUps != "amb", keep = TRUE)
```

Remove failed protein inference

Next, we remove PSMs that could not be mapped to a protein or that map to multiple proteins, i.e. a protein group. For the latter, the protein identifier contains multiple identifiers separated by a `;`). This information is readily available in the `rowData`, so there is no need for a custom filtering.

```
spikein <- filterFeatures(
    spikein, ~ Protein.Accessions != "" & ## Remove failed protein inference
        !grepl(";", Protein.Accessions)) ## Remove protein groups
```

Remove inconsistent protein inference

We also remove peptide ions that map to a different protein depending on the run. Again, this requires a custom filtering and we apply the same filtering workflow as above.

```
## 1. Collect data
rowdata <- rbindRowData(spikein, logNames)
## 2. Compute new variable
rowdata <- data.frame(rowdata) |>
    group_by(Annotated.Sequence, Charge) |>
    mutate(nProtsMapped = length(unique(Protein.Accessions)))
## 3. Reinsert in the rowData
rowData(spikein) <- split(rowdata, rowdata$assay)
## 4. Apply the filter
spikein <- filterFeatures(spikein, ~ nProtsMapped == 1, keep = TRUE)
```

Remove one-run wonders

We also remove proteins that can only be found in one run as such proteins may not be trustworthy. In this case,

```
## 1. Collect data
rowdata <- rbindRowData(spikein, logNames)
## 2. Compute new variable
rowdata <- data.frame(rowdata) |>
    group_by(Protein.Accessions) |>
```

```
    mutate(nRuns = length(unique(assay)))
## 3. Reinsert in the rowData
rowData(spikein) <- split(rowdata, rowdata$assay)
## 4. Apply the filter
spikein <- filterFeatures(spikein, ~ nRuns > 1, keep = TRUE)
```

Remove duplicated PSMs

Finally, peptide ions that were identified with multiple PSMs in a run are collapsed to the PSM with the highest summed intensity over the TMT labels, a strategy that is also used by MSstats.

This filtering requires a more complex workflow because it mixes information from the `rowData` (to obtain ion identities) with quantitative data (to obtain PSM intensity ranks). We therefore compute the filtering variable for every set iteratively:

1. Get the `rowData` for the current set.
2. Make a new variable `ionID`.
3. We calculate the `rowSums` for each ion.
4. Make a new variable `psmRank` that ranks the PSMs for each ion identifier based on the summed intensity.
5. We store the new information back in the `rowData`.

```
for (i in logNames) { ## for each set of interest
    rowdata <- rowData(spikein[[i]]) ## 1.
    rowdata$ionID <- paste0(rowdata$Annotated.Sequence, rowdata$Charge) ## 2.
    rowdata$rowSums <- rowSums(assay(spikein[[i]]), na.rm = TRUE) ## 3.
    rowdata <- data.frame(rowdata) |>
        group_by(ionID) |>
        mutate(psmRank = rank(-rowSums)) ## 4.
    rowData(spikein[[i]]) <- DataFrame(rowdata) ## 5.
}
```

For each ion that maps to multiple PSMs, we keep the PSM with the highest summed intensity, that is that ranks first.

```
spikein <- filterFeatures(spikein, ~ psmRank == 1, keep = TRUE)
```

Remove highly missing PSMs

We then remove PSMs that are missing in more than half of the 8 remaining TMT labels of a run. This is an arbitrary value that may need to be adjusted depending on the experiment and the data set.

```
spikein <- filterNA(spikein, logNames, pNA = 0.5)
```

Normalisation

As in the data processing chapter, we normalise the data with the median of ratios method. However, for TMT data we calculate the normalisation factors **for each run separately**:

1. Within a run, all TMT labels are quantified from the same MS2 spectra, so the PSMs of a run are observed in (nearly) all labels. The pseudo-reference (the mean of the log2 intensities of a PSM over the labels in the run) removes the PSM-specific intensity baseline and the run-specific spectrum effect, so that the median is taken over log2 ratios that vary much less than the log2 intensities themselves.
2. Between runs, the PSMs that are picked up differ and the same PSM can have a different intensity baseline. A pseudo-reference across runs therefore does not make sense. The differences in intensity level between runs are accounted for in the data modelling, with a random effect for run.

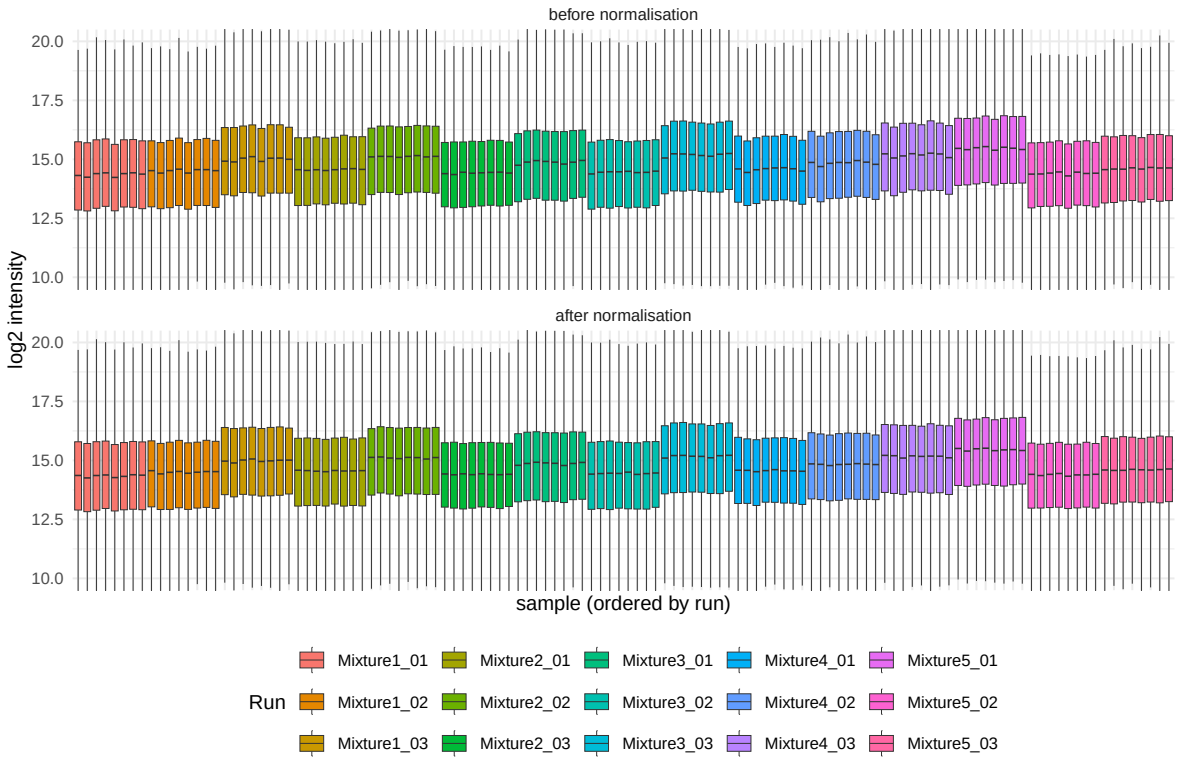
For each run, we compute the log2 normalisation factors with `nfLogMedianOfRatios()` and subtract them from the corresponding columns with `sweep()`.

```
normNames <- paste0(inputNames, "_norm")
for (k in seq_along(logNames)) {
    spikein <- sweep( # Subtract log2 norm factor column-by-column (MARGIN = 2)
        spikein,
        MARGIN = 2,
        STATS = nfLogMedianOfRatios(spikein, i = logNames[k]),
        i = logNames[k],
        name = normNames[k]
    )
}
```

We assess the impact of the normalisation with boxplots of the log2 intensities of all samples, before and after normalisation. The samples are ordered by run and coloured according to the run.

```r
intensities <- bind_rows(
    spikein[, , logNames] |>
        longForm(colvars = c("Run", "Mixture")) |>
        data.frame() |>
        mutate(step = "before normalisation"),
    spikein[, , normNames] |>
        longForm(colvars = c("Run", "Mixture")) |>
        data.frame() |>
        mutate(step = "after normalisation")) |>
    mutate(
        step = factor(step, levels = c("before normalisation", "after normalisation")),
        sample = paste(Run, sub("^.*Abundance\\.\\.", "", colname)),
        sample = factor(sample, levels = unique(sample)))

ggplot(intensities, aes(x = sample, y = value, fill = Run)) +
    geom_boxplot(outlier.shape = NA, linewidth = 0.2) +
    coord_cartesian(ylim = c(10, 20)) +
    facet_wrap(~ step, ncol = 1) +
    labs(x = "sample (ordered by run)", y = "log2 intensity") +
    theme_minimal() +
    theme(axis.text.x = element_blank(), legend.position = "bottom") +
    guides(fill = guide_legend(nrow = 3))
```

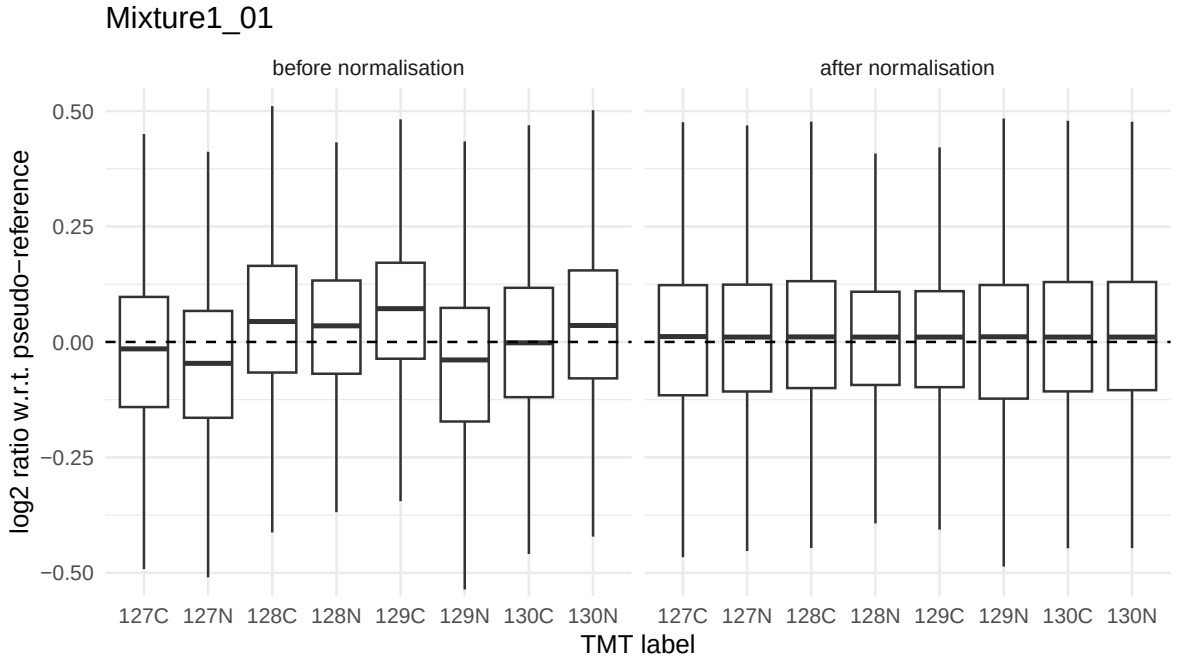


- Within a run, the normalisation factors are very small (the largest absolute log2 normalisation factor is 0.14): all labelled samples are pooled into a single TMT mixture before acquisition, so they are loaded in nearly equal amounts.
- The large differences in intensity are **between runs**, and even between runs of the same mixture. The median of ratios normalisation run by run does not remove these differences, because the pseudo-reference is calculated within each run. After normalisation, the intensity distributions are therefore only comparable within a run, not between runs.
- This is not a problem for the analysis: the summarisation is performed for each run separately and the model includes a random effect for run, which absorbs the run-specific intensity level.

The median of ratios method does not align the medians of the intensities, but the medians of the log2 ratios with respect to the pseudo-reference of the run. We illustrate this for the first run.

```
log2Ratios <- function(se) {
    y <- assay(se)
    y <- sweep(y, MARGIN = 1, STATS = rowMeans(y, na.rm = TRUE))
    colnames(y) <- sub("^.*Abundance\\.\\.", "", colnames(y))
    y
```

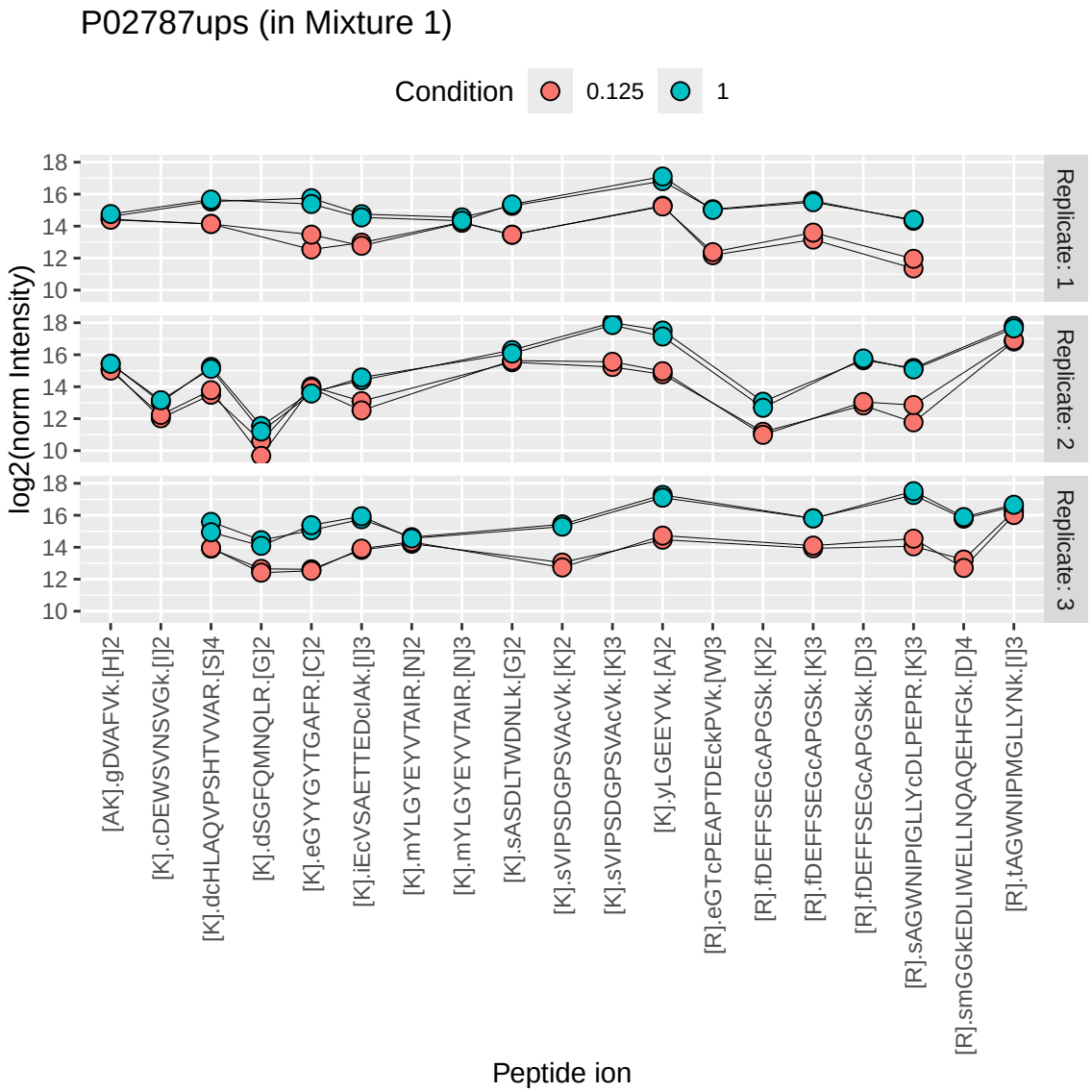
```
}
bind_rows(
    data.frame(log2Ratios(spikein[[logNames[1]]]), check.names = FALSE) |>
        mutate(step = "before normalisation"),
    data.frame(log2Ratios(spikein[[normNames[1]]]), check.names = FALSE) |>
        mutate(step = "after normalisation")) |>
    tidyr::pivot_longer(-step, names_to = "label", values_to = "log2ratio") |>
    mutate(step = factor(step, levels = c("before normalisation", "after normalisation"))) |>
    ggplot(aes(x = label, y = log2ratio)) +
    geom_boxplot(outlier.shape = NA) +
    geom_hline(yintercept = 0, linetype = "dashed") +
    coord_cartesian(ylim = c(-0.5, 0.5)) +
    facet_wrap(~ step) +
    labs(title = logNames[1] |> sub(pattern = "_log", replacement = ""),
         x = "TMT label", y = "log2 ratio w.r.t. pseudo-reference") +
    theme_minimal()
```



After normalisation, the median log2 ratios of all samples of the run are (nearly) equal.

Summarisation

Below, we illustrate the challenges of summarising TMT data using one of the UPS proteins in Mixture 1 (separating the data for each technical replicate). We also focus on the 0.125x and the 1x spike-in conditions. We illustrate the different peptide ions on the x axis and plot the log2 normalised intensities across samples on y axis. All the points belonging to the same sample are linked through a grey line.



We see that the same challenges observed for LFQ data also apply to TMT data. Briefly:

1. Data for a protein can consist of many peptide ions.

2. Peptide ions have different intensity baselines.
3. There is strong missingness across runs (compare points between replicates), but the missingness is mitigated within runs (compare points within replicates. Note that the data points from one peptide ion in one replicate has been extracted from a single MS2 spectrum.).
4. Subtle intensity shifts for the same peptide across different replicates, called spectrum effects, are caused by small run-to-run fluctuations.
5. Presence of outliers. For instance, the first peptide ion doesn't show the same change in intensity between conditions compared to majority of the peptides.

Here, we summarise the ion-level data into protein intensities with maxLFQ, as in the data processing chapter. maxLFQ first calculates all pairwise log2 ratios between samples using only the peptide ions they share, and takes the median of these ratios. The protein intensities are then obtained by least squares from these median pairwise ratios. Working with ratios removes the ion-specific baselines (issue 2.), and the median makes the summaries robust to outlying peptide ions (issue 5.).

The summarisation has to be performed for **each run separately**:

- the PSMs that are picked up and their intensity baseline differ from run to run, so the ion effects are estimated within each run, effectively allowing for a spectrum effect (issue 4.);
- within a run, the ions are observed in nearly all labels, so the ratios between the samples of a run are estimated precisely (issue 3.).

`aggregateFeatures()` summarises each run-specific set separately.

```
summNames <- paste0(inputNames, "_proteins")
(spikein <- aggregateFeatures(
    spikein, i = normNames, name = summNames,
    fcol = "Protein.Accessions",
    fun = function(X) iq::maxLFQ(X)$estimate
))
```

```
  |
  |                                                                      |   0%
  |
  |=====                                                                 |   7%
  |
  |=========                                                             |  13%
  |
  |==============                                                        |  20%
  |
```

```
  |==================                                                    |  27%
  |
  |=======================                                               |  33%
  |
  |===========================                                           |  40%
  |
  |================================                                      |  47%
  |
  |====================================                                  |  53%
  |
  |========================================                              |  60%
  |
  |============================================                          |  67%
  |
  |================================================                      |  73%
  |
  |============================================================          |  80%
  |
  |====================================================================  |  87%
  |
  |========================================================================  93%
  |
  |==========================================================================| 100%

An instance of class QFeatures (type: bulk) with 60 sets:

 [1] Mixture1_01: SummarizedExperiment with 1719 rows and 8 columns
 [2] Mixture1_02: SummarizedExperiment with 1722 rows and 8 columns
 [3] Mixture1_03: SummarizedExperiment with 1776 rows and 8 columns
 ...
 [58] Mixture5_01_proteins: SummarizedExperiment with 307 rows and 8 columns
 [59] Mixture5_02_proteins: SummarizedExperiment with 296 rows and 8 columns
 [60] Mixture5_03_proteins: SummarizedExperiment with 299 rows and 8 columns
```

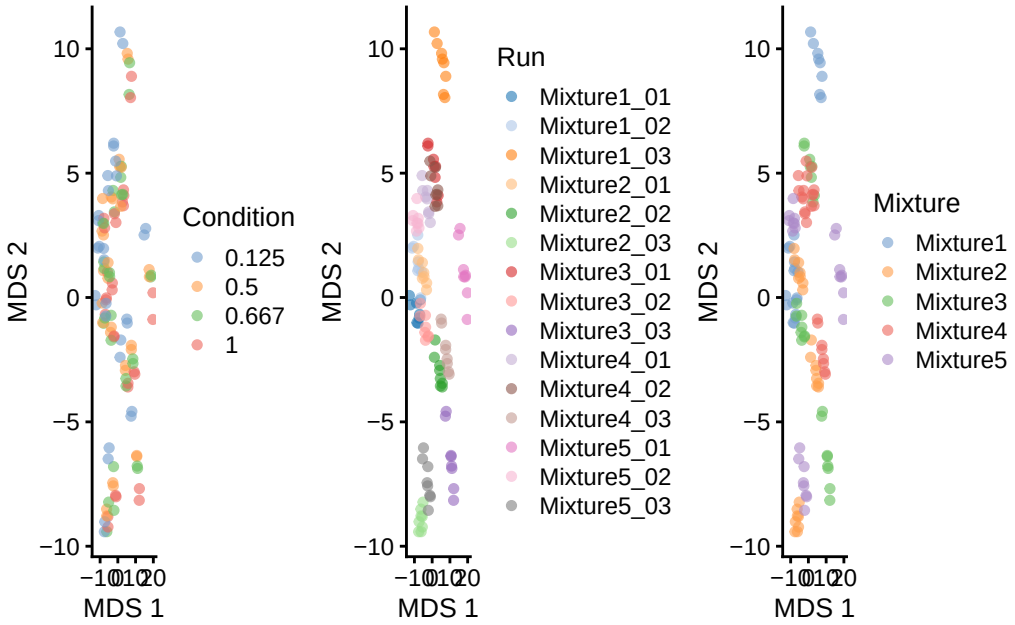
Up to now, the data from different runs were kept in separate assays. We can now join the normalised sets into an `proteins` set using `joinAssays()`. Sets are joined by stacking the columns (samples) in a matrix and rows (features) are matched according to the row names (i.e. the protein identifiers).

```
spikein <- joinAssays(
    spikein, summNames, "proteins"
)
```

Data exploration

We perform data exploration using MDS.

```
library("scater")
se <- getWithColData(spikein, "proteins")
se <- runMDS(as(se, "SingleCellExperiment"), exprs_values = 1)
plotMDS(se, colour_by = "Condition") +
  plotMDS(se, colour_by = "Run") +
  plotMDS(se, colour_by = "Mixture")
```



There is a strong run-to-run effect, which is partly explained by a mixture effect as the runs from the same mixture tend to be closer than runs from different mixtures. The condition effect is much more subtle to find, probably because we know only a few UPS proteins were spiked in while the majority of the background proteins are unchanged. We can see that normalisation and summarisation alone are not sufficient to correct for these unwanted effects. We will take care of these effects during the data modelling.

Data modelling

Proteomics data contain several sources of variation that need to be accounted for by the model. We will build the model by progressively adding the different sources of variation, following the

msqrob2TMT modelling strategy of Vandenbulcke et al. (2025).

Effect of treatment of interest

We model the source of variation induced by the experimental treatment of interest as a **fixed effect**, which we consider non-random, i.e. the treatment effect is assumed to be the same in repeated experiments, but it is unknown and has to be estimated. When modelling a typical label-free experiment at the protein level, the model boils down to a linear model, again we suppress the index for protein:

$$y_r = \mathbf{x}_r^T \beta + \epsilon_r,$$

with y_r the $\log_2$-normalized protein intensities in run r; $\mathbf{x}_r$ a vector with the covariate pattern for the sample in run r encoding the intercept, treatment, potential batch effects and confounders; β the vector of parameters that model the association between the covariates and the outcome; and ϵ_r the residuals reflecting variation that is not captured by the fixed effects. Note that $\mathbf{x}_r$ allows for a flexible parameterization of the treatment beyond a single covariate, i.e. including a 1 for the intercept, continuous and categorical variables as well as their interactions. For all models considered in this work, we assume the residuals to be independent and identically distributed (i.i.d) according to a normal distribution with zero mean and constant variance, i.e. $\epsilon_r \sim N(0, \sigma_\epsilon^2)$, that can differ from protein to protein.

Now we defined a model, we must estimate from the data. Using `msqrob()`, the model translates into the following code:

```
spikein <- msqrob(
    spikein,  i = "proteins",
    formula = ~  Condition ## fixed effect for experimental condition
)
```

This model is however incomplete and relying on its results would lead to incorrect conclusions as we are still missing important source of variation. We did therefore not run the modelling command and will expand the model.

Effect of TMT label and run

As label-free experiments contain only a single sample per run, run-specific effects will be absorbed in the residuals. However, the data analysis of labeled experiments, e.g. using TMT multiplexing, involving multiple MS runs has to account for run- and label-specific effects, explicitly. If all treatments are present in each run, and if TMT label swaps are performed so as to avoid confounding between label and treatment, then the model parameters can be

estimated using fixed label and run effects. Indeed, for these designs run acts as a blocking variable as all treatment effects can be estimated within each run.

However, for more complex designs this is no longer possible and the uncertainty in the estimation of the mean model parameters can involve both within and between TMT label and run variability. For these designs we can resort to mixed models where the label and run effect are modelled using **random effects**, i.e. they are considered as a random sample from the population of all possible runs and TMT labels, which are assumed to be i.i.d normally distributed with mean 0 and constant variance, $b_r^{\text{run}} \sim N(0, \sigma^2_{\text{run}})$ ($b_l^{\text{label}} \sim N(0, \sigma^2_{\text{label}})$). The use of random effects thus models the correlation in the data, explicitly. Indeed, protein intensities that are measured within the same run/label will be more similar than protein intensities between runs/labels.

Hence, the model is extended to:

$$y_{rl} = \mathbf{x}_{rl}^T \beta + b_l^{\text{label}} + b_r^{\text{run}} + \epsilon_{rl}$$

with y_{rl} the normalised $\log_2$ protein intensities in run r and label l, b_l^{label} the effect introduced by the label l, and b_r^{run} the effect for MS run r.

This translates in the following code:

```
spikein <- msqrob(
    spikein,  i = "proteins",
    formula = ~  Condition + ## fixed effect for experimental condition
        (1 | Label) + ## random effect for label
        (1 | Run) ## random effect for MS run
    )
```

This model is still incomplete and is not executed, as we still need to account for the fact that each mixture has been acquired three times.

Effect of mixture

Each TMT mixture has been acquired in three technical replicate runs. Samples from the same mixture share the same sample handling after labelling, e.g. the pooling of the labelled samples. Protein intensities from the same mixture will therefore be more alike than those of different mixtures. We account for this correlation with a random effect for mixture, $b_m^{\text{mix}} \sim N(0, \sigma^2_{\text{mix}})$.

Effect of sample: technical replication

Each mixture has been acquired in **three runs**, so every labelled sample, i.e. a combination of a mixture m and a TMT label l, has been measured three times. Note that a label alone does not identify a sample: label 126 in mixture 1 and label 126 in mixture 2 are different samples.

- The spike-in condition is assigned to the samples: the **sample** is the experimental unit.
- The three runs of the same mixture are technical repeats of the same samples: the measurements are the observational units and they are **pseudo-replicates**.

This is exactly the situation of the fish in a tank, or of the technical repeats in the Francisella exercise of the design tutorial. In terms of the random treatment effects in block designs: each run and each mixture is a block in which all conditions are present, and each sample (mixture $\times$ label) is measured repeatedly within the block, i.e. we have technical repeats of each block $\times$ treatment combination. A random effect for sample, $b_{ml}^{\text{sample}} \sim N(0, \sigma^2_{\text{sample}})$, is then always needed.

- The condition effect is estimated within runs and mixtures, so the run and mixture effects cancel in the comparison, as in a blocked design.
- However, the condition effect is a comparison between different samples. Its uncertainty therefore involves the variability between samples. Without the sample effect, the model would treat the three runs of the same sample as three independent samples, i.e. 30 instead of 10 samples per condition (5 mixtures $\times$ 2 labels). The between-sample variability would then end up in the residuals and be averaged over too many measurements, leading to too small standard errors and too many false positives.
- In this spike-in study the samples are technical dilutions of the same material, so the between-sample variability will be small. In experiments with biological samples, it also contains the biological variability between the samples, and ignoring it is much more harmful.

Each condition is present in two samples of each mixture. We could therefore also add a random mixture $\times$ condition interaction, `(1 | Mixture:Condition)`, which is needed when the condition effect varies from mixture to mixture. For a spike-in study with known dilutions we do not expect such an interaction, so we do not include it here.

The model thus extends to:

$$y_{rlm} = \mathbf{x}_{rlm}^T \beta + b_l^{\text{label}} + b_r^{\text{run}} + b_m^{\text{mix}} + b_{ml}^{\text{sample}} + \epsilon_{rlm}$$

with m the index for mixture.

The model translates to the following code:

```
spikein <- msqrob(
    spikein,  i = "proteins",
    formula = ~  Condition + ## fixed effect for experimental condition
        (1 | Label) + ## random effect for label
        (1 | Run) + ## random effect for MS run
        (1 | Mixture) + ## random effect for mixture
        (1 | Mixture:Label) ## random effect for sample
)
```

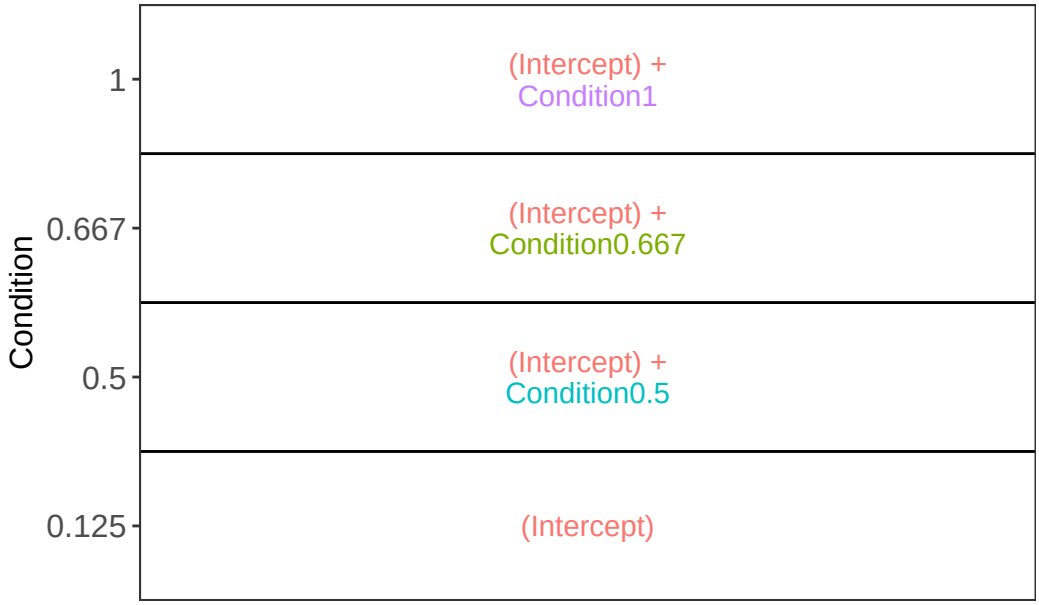
This model provides a sensible representation of the sources of variation in the data.

Statistical inference

We can now convert the biological question "does the spike-in condition affect the protein intensities?" into a statistical hypothesis. In other words, we need to define the contrast. We plot an overview of the model parameters.

```
library("ExploreModelMatrix")
vd <- VisualizeDesign(
    sampleData =  colData(spikein),
    designFormula = ~ Condition,
    textSizeFitted = 4
)
vd$plotlist
```

```
[[1]]
```



Hypothesis testing

The average difference intensity between the 1x and the 0.5x conditions is provided by the contrast `Condition1 - Condition0.5`.

```
hypothesis <- "Condition1 - Condition0.5 = 0"
L <- makeContrast(
    hypothesis,
    parameterNames = c("Condition1", "Condition0.5")
)
```

We test our null hypotheses using `hypothesisTest()` and the estimated model stored in `proteins`.

```
spikein <- hypothesisTest(spikein, i = "proteins", contrast = L)
inference <- rowData(spikein[["proteins"]])[, colnames(L)]
head(inference)
```

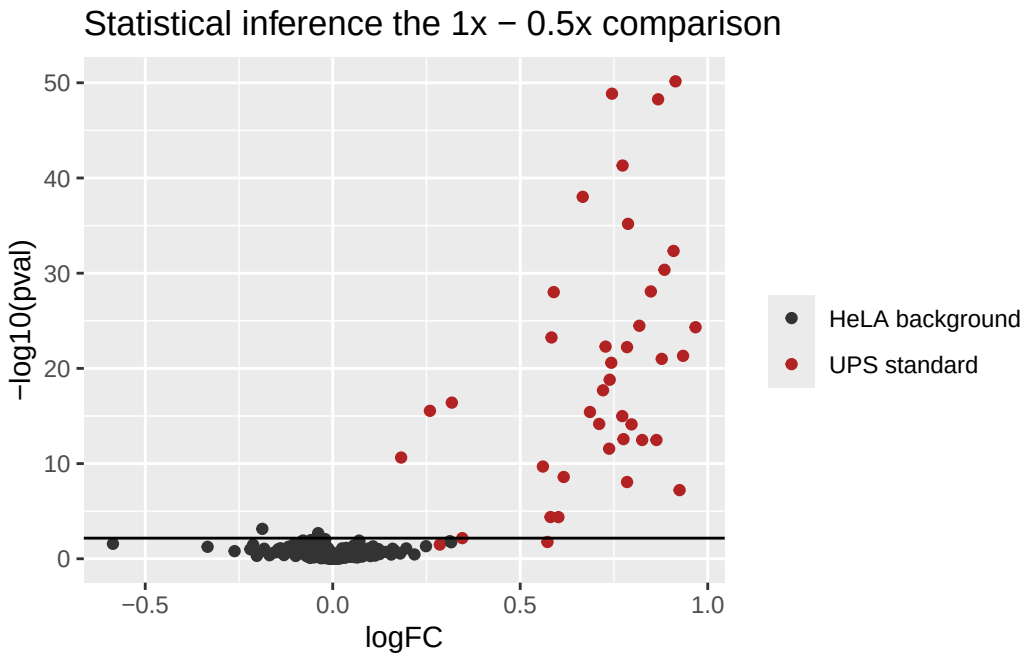
```
              logFC          se        df          t       pval   adjPval
O00151  0.007167648 0.024323487 101.58871  0.2946801 0.76883960 0.9747322
O00299 -0.012174869 0.011531707  95.70637 -1.0557733 0.29372963 0.7645183
```

```
O00410  0.002934538 0.008978378 103.78402  0.3268450 0.74444340 0.9747322
O00629  0.012508367 0.023658911 103.85166  0.5286958 0.59814460 0.9419336
O14745 -0.024664939 0.013689786 103.80567 -1.8017038 0.07449477 0.3665927
O14980 -0.012971894 0.008173085 102.87363 -1.5871479 0.11554818 0.4485244
```

Volcano plots

We generate a volcano plot with all the results for the 1- 0.5x comparison.

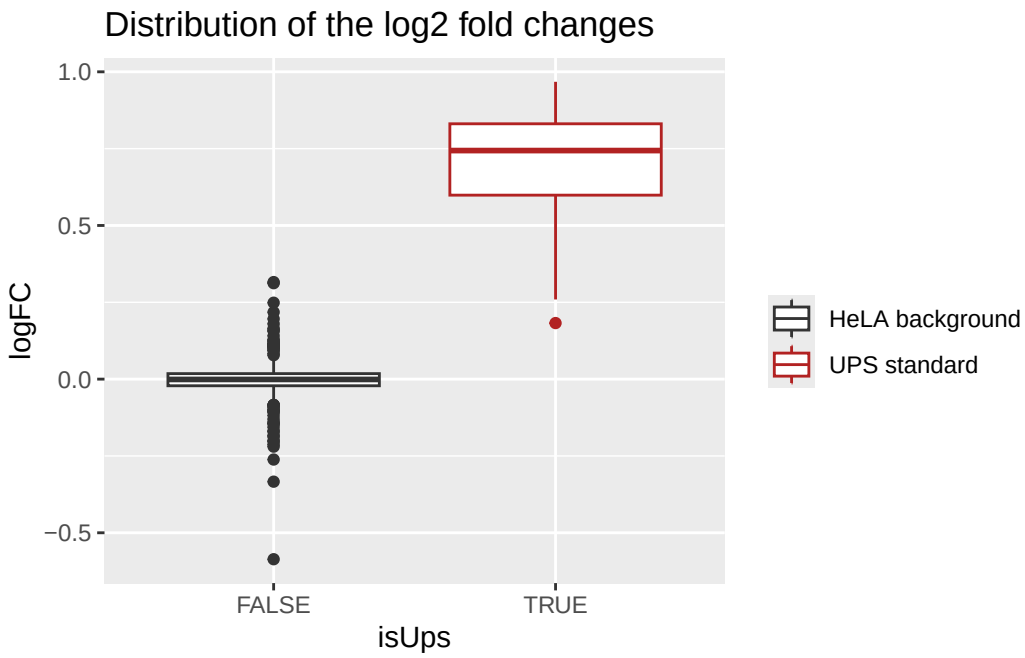
```
inference$Protein <- rownames(inference)
inference$isUps <- grepl("ups", inference$Protein)
ggplot(inference) +
    aes(x = logFC,
        y = -log10(pval),
        color = isUps) +
    geom_point() +
    geom_hline(yintercept = -log10(inference$pval[which.min(abs(inference$adjPval - 0.05))] )
    scale_color_manual(
      values = c("grey20", "firebrick"), name = "",
      labels = c("HeLA background", "UPS standard")
    ) +
    ggtitle("Statistical inference the 1x - 0.5x comparison")
```



Fold change distributions

As this is a spike-in study with known ground truth, we can also plot the log2 fold change distributions against the expected values, in this case 0 for the HeLa proteins and 1 for the UPS standards.

```
ggplot(inference) +
    aes(y = logFC,
        x = isUps,
        colour = isUps) +
    geom_boxplot() +
    scale_color_manual(
        values = c("grey20", "firebrick"), name = "",
        labels = c("HeLA background", "UPS standard")
    ) +
    ggtitle("Distribution of the log2 fold changes")
```



The estimated log2 fold changes for HeLa proteins are closely distributed around 0, as expected. log2 fold changes for UPS standard proteins are distributed toward 1, but it has been underestimated due to ion suppression effects that are characteristic of data set with high spike-in concentrations.

Exercise

In the design tutorial you will apply this workflow to a real-life TMT experiment on the effect of diet on the adipose tissue proteome of mice.

Session Info

With respect to reproducibility, it is highly recommended to include a session info in your script so that readers of your output can see your particular setup of R.

```
sessionInfo()
```

```
R version 4.6.1 (2026-06-24)
Platform: aarch64-apple-darwin23
Running under: macOS Tahoe 26.1

Matrix products: default
BLAS:   /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRlapack.dylib;  LAPACK

locale:
[1] nl_BE.UTF-8/nl_BE.UTF-8/nl_BE.UTF-8/C/nl_BE.UTF-8/nl_BE.UTF-8

time zone: Europe/Brussels
tzcode source: internal

attached base packages:
[1] stats4    stats     graphics  grDevices utils     datasets  methods
[8] base

other attached packages:
 [1] ExploreModelMatrix_1.24.0   scater_1.40.1
 [3] scuttle_1.22.0              SingleCellExperiment_1.34.0
 [5] BiocFileCache_3.2.0         dbplyr_2.6.0
 [7] BiocParallel_1.46.0         patchwork_1.3.2
 [9] ggplot2_4.0.3               dplyr_1.2.1
[11] msqrob2_1.21.1              QFeatures_1.22.0
[13] MultiAssayExperiment_1.38.0 SummarizedExperiment_1.42.0
[15] Biobase_2.72.0              GenomicRanges_1.64.0
[17] Seqinfo_1.2.0               IRanges_2.46.0
```

```
[19] S4Vectors_0.50.1           BiocGenerics_0.58.1
[21] generics_0.1.4             MatrixGenerics_1.24.0
[23] matrixStats_1.5.0

loaded via a namespace (and not attached):
  [1] RColorBrewer_1.1-3      jsonlite_2.0.0          magrittr_2.0.5
  [4] estimability_2.0.0      ggbeeswarm_0.7.3        farver_2.1.2
  [7] nloptr_2.2.1            rmarkdown_2.31          vctrs_0.7.3
 [10] memoise_2.0.1           minqa_1.2.8             htmltools_0.5.9
 [13] S4Arrays_1.12.0         BiocBaseUtils_1.14.2    curl_7.1.0
 [16] BiocNeighbors_2.6.0     SparseArray_1.12.2      htmlwidgets_1.6.4
 [19] plyr_1.8.9              httr2_1.2.3             cachem_1.1.0
 [22] igraph_2.3.3            mime_0.13               lifecycle_1.0.5
 [25] pkgconfig_2.0.3         rsvd_1.0.5              Matrix_1.7-5
 [28] R6_2.6.1                fastmap_1.2.0           rbibutils_2.4.1
 [31] shiny_1.14.0            clue_0.3-68             digest_0.6.39
 [34] irlba_2.3.7             RSQLite_3.53.2          beachmat_2.28.0
 [37] filelock_1.0.3          labeling_0.4.3          abind_1.4-8
 [40] compiler_4.6.1          bit64_4.8.2             withr_3.0.3
 [43] S7_0.2.2                viridis_0.6.5           DBI_1.3.0
 [46] MASS_7.3-65             iq_2.0.1                rappdirs_0.3.4
 [49] DelayedArray_0.38.2     tools_4.6.1             vipor_0.4.7
 [52] otel_0.2.0              beeswarm_0.4.0          httpuv_1.6.17
 [55] glue_1.8.1              nlme_3.1-169            promises_1.5.0
 [58] grid_4.6.1              cluster_2.1.8.2         reshape2_1.4.5
 [61] gtable_0.3.6            tidyr_1.3.2             BiocSingular_1.28.0
 [64] ScaledMatrix_1.20.0     XVector_0.52.0          ggrepel_0.9.8
 [67] pillar_1.11.1           stringr_1.6.0           limma_3.68.4
 [70] later_1.4.8             rintrojs_0.3.4          splines_4.6.1
 [73] lattice_0.22-9          bit_4.6.0               tidyselect_1.2.1
 [76] knitr_1.51              reformulas_0.4.4        gridExtra_2.3.1
 [79] ProtGenerics_1.44.0     shinydashboard_0.7.3    xfun_0.59
 [82] statmod_1.5.2           DT_0.34.0               stringi_1.8.7
 [85] lazyeval_0.2.3          yaml_2.3.12             boot_1.3-32
 [88] evaluate_1.0.5          codetools_0.2-20        MsCoreUtils_1.24.0
 [91] tibble_3.3.1            cli_3.6.6               xtable_1.8-8
 [94] Rdpack_2.6.6            Rcpp_1.1.1-1.1          parallel_4.6.1
 [97] assertthat_0.2.1        blob_1.3.0              AnnotationFilter_1.36.0
[100] lme4_2.0-1              viridisLite_0.4.3       scales_1.4.0
[103] purrr_1.2.2             rlang_1.2.0             cowplot_1.2.0
[106] shinyjs_2.1.1
```

References

Huang, Ting, Meena Choi, Manuel Tzouros, et al. 2020. "MSstatsTMT: Statistical Detection of Differentially Abundant Proteins in Experiments with Isobaric Labeling and Multiple Mixtures." *Mol. Cell. Proteomics* 19 (10): 1706–23.

Vandenbulcke, Stijn, Christophe Vanderaa, Oliver Crook, Lennart Martens, and Lieven Clement. 2025. "Msqrob2TMT: Robust Linear Mixed Models for Inferring Differential Abundant Proteins in Labeled Experiments with Arbitrarily Complex Design." *Mol. Cell. Proteomics* 24 (7): 101002.